

-  Secrets
-  ABAP
-  Ansible
-  Apex
-  AzureResourceManager
-  C
-  C#
-  C++
-  CloudFormation
-  COBOL
-  CSS
-  **Dart**
-  Docker
-  Flex
-  Go
-  HTML
-  Java
-  JavaScript
-  JCL
-  Kotlin
-  Kubernetes
-  Objective C
-  PHP
-  PL/I
-  PL/SQL
-  Python
-  RPG
-  Ruby
-  Scala
-  Swift
-  Terraform
-  Text
-  TypeScript
-  T-SQL
-  VB.NET
-  VB6
-  XML



Dart static code analysis

Unique rules to write Clean DART Code

All rules 115

 Bug 15

 Code Smell 100

Tags

▼

Impact

▼

Clean code attribute

▼

Search by name...

🔍

"isEmpty" or "isNotEmpty" should be used to test for emptiness

 Code Smell

Unnecessary imports should be removed

 Code Smell

Empty statements should be removed

 Code Smell

Class names should comply with a naming convention

 Code Smell

Track uses of "TODO" tags

 Code Smell

Deprecated code should be removed

 Code Smell

Cyclomatic Complexity of functions should not be too high

 Code Smell

Constant names should comply with a naming convention

 Code Smell

Unnecessary nullable in final declaration should be removed

 Code Smell

Dependencies should be sorted

 Code Smell

Unused assignments should be removed

 Code Smell

"isEmpty" or "isNotEmpty" should be used to test for emptiness

Analyze your code

Intentionality - Clear

Maintainability 👍

 Code Smell

 Minor ?

 clumsy

Why is this an issue?

More Info

When you call `isEmpty` or `isNotEmpty`, it clearly communicates the code's intention, which is to check if the collection is empty. Using `.length == 0` for this purpose is less direct and makes the code slightly more complex.

The rule also raises issues if the comparisons don't make sense. For example, `length` is always 0 or higher, so you don't need to write the following conditions:

```
void fun(List<int> myList) {
  if (myList.length >= 0) { // Noncompliant, the condition is always true
    // do something
  }

  if (myList.length < 0) { // Noncompliant, the condition is always false
    // do something
  }
}
```

Code examples

Noncompliant code example

```
void fun(List<int> myList) {
  if (myList.length == 0) { // Noncompliant
    // do something
  }

  if (myList.length != 0) { // Noncompliant
    // do something
  }
}
```

Compliant solution

```
void fun(List<int> myList) {
  if (myList.isEmpty) {
    // do something
  }

  if (myList.isNotEmpty) {
    // do something
  }
}
```

Available In:

sonarcloud 

